

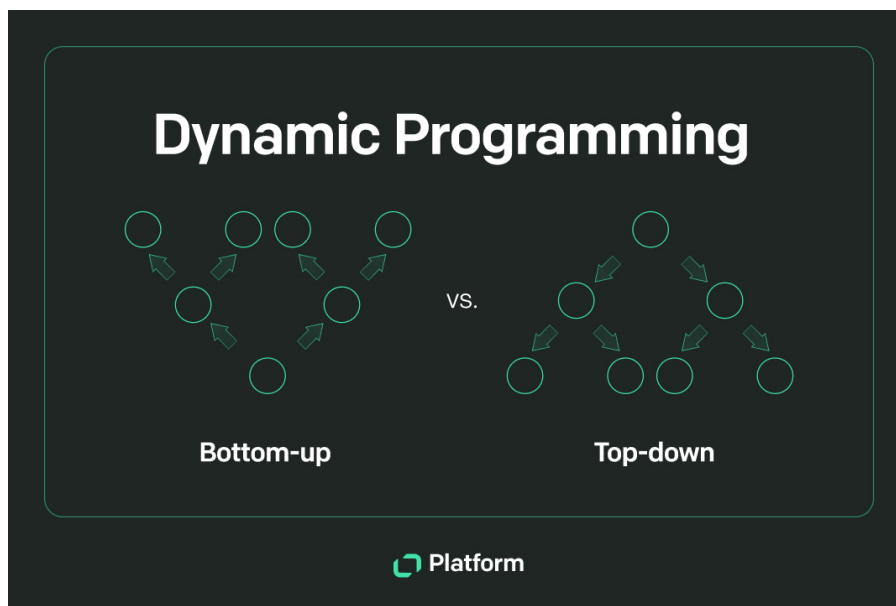
LAB 6: DYNAMIC PROGRAMMING

1. Introduction to Dynamic Programming

1.1. Definition

Dynamic Programming (DP) is an optimization method that breaks a large problem into smaller subproblems, solves the subproblems, and stores the results to avoid redundant computations. This technique is particularly useful for problems with a recursive structure, where solving the same subproblem multiple times would reduce efficiency.

Unlike the Divide and Conquer approach, where each subproblem is solved independently, Dynamic Programming leverages stored results from subproblems to minimize redundant calculations.



There are two main approaches in Dynamic Programming:

- **Top-down (Memoization):** Uses recursion with stored results to prevent redundant computations.
- **Bottom-up (Tabulation):** Builds solutions from the smallest subproblems first, then combines them to find the optimal solution for the larger problem.

1.2. Fundamental Principles

Dynamic Programming is based on two key principles:

❖ Optimal Substructure

A problem exhibits an optimal substructure if an optimal solution to the problem can be constructed from optimal solutions to its subproblems. This means we can divide the problem into smaller components, solve each part, and combine the results to obtain the final solution.

Examples:

- In the Longest Common Subsequence (LCS) problem, if the last characters of two sequences match, the solution can be derived from the LCS of the two smaller sequences.
- In the Knapsack problem, the optimal value when considering items can be determined from the solution with items.

❖ Overlapping Subproblems

A problem exhibits overlapping subproblems if the same subproblem appears multiple times in the computation. If these subproblems are not stored, redundant computations can lead to inefficiency.

Examples:

- In the Fibonacci sequence, a naive recursive approach recalculates the same Fibonacci numbers multiple times, resulting in a time complexity of $O(2^n)$. Using Dynamic Programming to store results can reduce the complexity to $O(n)$.
- By storing the results of subproblems in a table (array or other data structures), we can retrieve previously computed values without redundant calculations, significantly improving efficiency.

1.3. Comparison of Top-down and Bottom-up Approaches

Approach	Description	Pros	Cons
Top-down	Uses recursion and	- Easier to implement,	- Can be slow due to

(Memoization)	stores results to avoid redundant calculations.	intuitive for recursive problems. - State transition is easy to create - Code is simpler and easier to understand.	recursive calls and return statements. - High recursion overhead (stack usage). - Entries in the table are filled on demand.
Bottom-up (Tabulation)	Iteratively builds the solution from smaller subproblems.	- Avoids recursion overhead, often more efficiently. - Fast due to direct access to the table of results of previously solved subproblems. - Works well for problems with clear state transitions.	- State transition can be difficult to define. - If many conditions are required, code can be complex. - All entries must be filled sequentially.

1.4. Four steps of Dynamic Programming Algorithm

When designing a Dynamic programming algorithm, we follow four key steps:

Step 1: Characterize the Structure of an Optimal Solution

Step 2: Recursively define the value of an optimal solution.

Step 3: Compute the value of an optimal solution, in a bottom-up or top-down fashion.

Step 4: Construct an optimal solution from computed information.

2. Classification of Dynamic Programming Problems

Programming problems can be categorized into two main types:

❖ Optimization Problems

- Rod Cutting Problem
- Knapsack Problem
- Longest Common Subsequence (LCS)

- Minimum Spanning Tree (MST)

❖ Counting Problems

- Shortest Path (Floyd-Warshall, Dijkstra)
- Counting Paths in a Grid
- Coin Change Problem

3. Detailed of some Dynamic Programming Problems

3.1. Cutting Rod Problem

Problem statement

Given a rod of length n inches and a table of prices p_i for $i = 1; 2; \dots; n$, determine the maximum revenue r^n obtainable by cutting up the rod and selling the pieces. Note that if the price p^n for a rod of length n is large enough, an optimal solution may require no cutting at all. Consider the case when $n = 4$. Figure 2 shows all the ways to cut up a rod of 4 inches in length, including the way with no cuts at all. We see that cutting a 4-inch rod into two 2-inch pieces produces revenue $p_2 + p_2 = 5 + 5 = 10$, which is optimal.

Length i	1	2	3	4	5	6	7	8	9	10
Price p_i	1	5	8	9	10	17	17	20	24	30

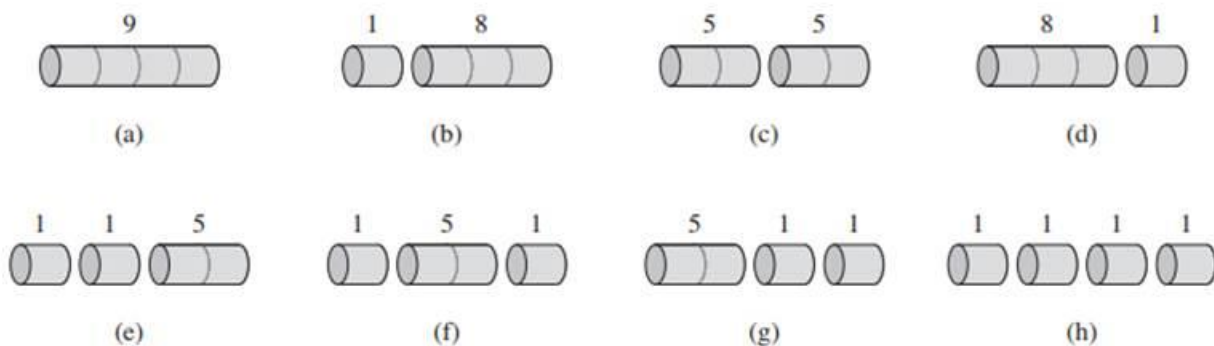


Figure 2: The 8 possible ways of cutting up a rod of length 4

If an optimal solution cuts the rod into k pieces, for some $1 \leq k \leq n$, then an optimal decomposition:

$$n = i_1 + i_2 + \dots + i_k \quad (1)$$

of the rod into pieces of lengths $i_1, i_2, \dots, i_k$ provides maximum corresponding revenue

$$r_n = p_{i_1} + p_{i_2} + \dots + p_{i_k} \quad (2)$$

For our sample problem, we can determine the optimal revenue figures r_i , for $i = 1; 2; \dots; 10$, by inspection, with the corresponding optimal decompositions:

$r_1 = 1$ from solution $1 = 1$ (no cuts)

$r_2 = 5$ from solution $2 = 2$ (no cuts)

$r_3 = 8$ from solution $3 = 3$ (no cuts)

$r_4 = 10$ from solution $4 = 2 + 2$

$r_5 = 13$ from solution $5 = 2 + 3$

$r_6 = 17$ from solution $6 = 6$

$r_7 = 18$ from solution $7 = 1 + 6$ or $7 = 2 + 2 + 3$

$r_8 = 22$ from solution $8 = 2 + 6$

$r_9 = 25$ from solution $9 = 3 + 6$

$r_{10} = 30$ from solution $10 = 10$

We view a decomposition as consisting of a first piece of length i cut off the left-hand end, and then a right-hand remainder of length $n - i$. Only the remainder, and not the first piece, may be further divided. We may view every decomposition of a length- n rod in this way: as a first piece followed by some decomposition of the remainder. When doing so, we can couch the solution with no cuts at all as saying that the first piece has size $i = n$ and revenue p_n and that the remainder has size 0 with corresponding revenue $r_0 = 0$. We can frame the values r_n for $n \geq 1$ in terms of optimal revenues from shorter rods:

$$r_n = \max(p_i + r_{n-i}), 1 \leq i \leq n \quad (3)$$

Recursive top-down implementation

The following procedure implements the computation implicit in equation (3) in a straightforward, top-down, recursive manner.

```

CUT-ROD( $p, n$ )
1  if  $n == 0$ 
2      return 0
3   $q = -\infty$ 
4  for  $i = 1$  to  $n$ 
5       $q = \max(q, p[i] + \text{CUT-ROD}(p, n - i))$ 
6  return  $q$ 

```

However, this is an inefficient algorithm. The problem is that CUT-ROD calls itself recursively over and over again with the same parameter values; it solves the same subproblems repeatedly. To solve this problem, we use dynamic programming for optimal rod cutting to ensure that each subproblem to be solved only *once*.

Top-down approach

```

MEMOIZED-CUT-ROD-AUX( $p, n, r$ )
1  if  $r[n] \geq 0$ 
2      return  $r[n]$ 
3  if  $n == 0$ 
4       $q = 0$ 
5  else  $q = -\infty$ 
6      for  $i = 1$  to  $n$ 
7           $q = \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$ 
8   $r[n] = q$ 
9  return  $q$ 

```

Here, the main procedure MEMOIZED-CUT-ROD initializes a new auxiliary array $r[0 \dots n]$ with the value $-\infty$, a convenient choice with which to denote “unknown.” (Known revenue values are always nonnegative.) It then calls its helper routine, MEMOIZED-CUT-ROD-AUX.

The procedure MEMOIZED-CUT-ROD-AUX is just the memoized version of our previous procedure, CUT-ROD. It first checks in line 1 to see whether the desired value is already known and, if it is, then line 2 returns it. Otherwise, lines 3–7 compute the desired value q in the usual manner, line 8 saves it in $r[n]$, and line 9 returns it.

Bottom-up approach

```
BOTTOM-UP-CUT-ROD( $p, n$ )
1  let  $r[0 \dots n]$  be a new array
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6           $q = \max(q, p[i] + r[j - i])$ 
7       $r[j] = q$ 
8  return  $r[n]$ 
```

For the bottom-up dynamic-programming approach, BOTTOM-UP-CUT-ROD uses the natural ordering of the subproblems: a problem of size i is “smaller” than a subproblem of size j if $i < j$. Thus, the procedure solves subproblems of sizes $j = 0; 1; \dots; n$, in that order.

Line 1 of procedure BOTTOM-UP-CUT-ROD creates a new array $r[0 \dots n]$ in which to save the results of the subproblems, and line 2 initializes $r[0]$ to 0, since a rod of length 0 earns no revenue. Lines 3–6 solve each subproblem of size j , for $j = 1; 2; \dots; n$, in order of increasing size. The approach used to solve a problem of a particular size j is the same as that used by CUT-ROD, except that line 6 now directly references array entry $r[j-i]$ instead of making a recursive call to solve the subproblem of size $j - i$. Line 7 saves in $r[j]$ the solution to the subproblem of size j . Finally, line 8 returns $r[n]$, which equals the optimal value r_n .

Reconstructing a solution

We can extend the dynamic-programming approach to record not only the optimal *value* computed for each subproblem, but also a *choice* that led to the optimal value. With this information, we can readily print an optimal solution.

Here is an extended version of BOTTOM-UP-CUT-ROD that computes, for each rod size j , not only the maximum revenue r_j , but also s_j , the optimal size of the first piece to cut off:

EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

```
1  let  $r[0..n]$  and  $s[0..n]$  be new arrays
2   $r[0] = 0$ 
3  for  $j = 1$  to  $n$ 
4       $q = -\infty$ 
5      for  $i = 1$  to  $j$ 
6          if  $q < p[i] + r[j - i]$ 
7               $q = p[i] + r[j - i]$ 
8               $s[j] = i$ 
9       $r[j] = q$ 
10 return  $r$  and  $s$ 
```

This procedure is similar to BOTTOM-UP-CUT-ROD, except that it creates the array s in line 1, and it updates $s[j]$ in line 8 to hold the optimal size i of the first piece to cut off when solving a subproblem of size j .

The following procedure takes a price table p and a rod size n , and it calls EXTENDED-BOTTOM-UP-CUT-ROD to compute the array $s[1 \dots n]$ of optimal first-piece sizes and then prints out the complete list of piece sizes in an optimal decomposition of a rod of length n :

PRINT-CUT-ROD-SOLUTION(p, n)

```
1  ( $r, s$ ) = EXTENDED-BOTTOM-UP-CUT-ROD( $p, n$ )
2  while  $n > 0$ 
3      print  $s[n]$ 
4       $n = n - s[n]$ 
```

In our rod-cutting example, the call EXTENDED-BOTTOM-UP-CUT-ROD($p, 10$) would return the following arrays:

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

A call to PRINT-CUT-ROD-SOLUTION($p, 10$) would print just 10, but a call with $n = 7$ would print the cuts 1 and 6, corresponding to the first optimal decomposition for r_7 given earlier.

Example code for top-down approach

```

#include <stdio.h>
#include <limits.h>

// Top-down memoized cut rod function
int memoized_cut_rod_aux(int p[], int n, int r[], int s[]) {
    if (r[n] >= 0)
        return r[n];

    if (n == 0)
        q = 0;
    else
        q = -1;

    for (int i = 1; i <= n; i++) {
        int temp = p[i] + memoized_cut_rod_aux(p, n - i, r, s);
        if (temp > q) {
            q = temp;
            s[n] = i; // Store optimal first cut
        }
    }
    r[n] = q;
    return q;
}

// Function to print rod cutting solution
void print_cut_rod_solution(int s[], int n) {
    while (n > 0) {
        printf("%d ", s[n]);
        n -= s[n];
    }
}

int main() {
    int price[] = {0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30};
    int n = 10; // Rod length

    int r[n + 1], s[n + 1];
    for (int i = 0; i <= n; i++)
        r[i] = -1; // Initialize revenues to -1 to mark "unkown"

    int max_revenue = memoized_cut_rod_aux(p, n, r, s);

    printf("Optimal revenue: %d\n", max_revenue);
    printf("Cuts: ");
    print_cut_rod_solution(s, n);
    printf("\n");
}

```

```
return 0;
```

3.2. Knapsack Problem

Problem Given n items, where each item i has:

- Weight: w_i
- Value: v_i
- A knapsack with a maximum capacity of W .

Objective Select a subset of items such that:

- The total weight does not exceed W .
- The total value of the selected items is maximized.

Constraint

Each item can either be taken completely or not taken at all (0/1). Partial selection is not allowed.

State Definition

Define the state function: $dp(i, w)$

where $dp(i, w)$ represents the maximum value achievable by considering the first i items with a knapsack capacity of w .

Example:

$$dp(2, 30) = 100$$

This means that when considering items 1 and 2 with a knapsack capacity of 30, the maximum possible value is 100.

Recurrence Relation

Case 1: Not selecting item i

If we do not select item i , the total value remains the same as when considering $i-1$ items:

$$dp(i, w) = dp(i-1, w)$$

Case 2: Selecting item i

The condition to select item i is that its weight must not exceed the knapsack capacity:

$$w_i \leq w$$

If we select item i , we:

- Reduce the knapsack capacity by w_i
- Add the item's value v_i to the optimal value from the previous state.

$$dp(i, w) = dp(i-1, w-w_i) + v_i$$

Combining both cases:

$$dp(i, w) = \begin{cases} dp(i-1, w) & \text{(if item } i \text{ is not selected)} \\ \max(dp(i-1, w), dp(i-1, w-w_i) + v_i) & \text{(if item } i \text{ is selected and } w_i \leq w) \end{cases}$$

Interpretation: When considering item i , we choose the maximum between:

- Not selecting the item (keeping the previous value).
- Selecting the item (adding its value and reducing the capacity accordingly).

Base Cases

- If there are no items ($i=0$), the optimal value is always 0:

$$dp(0, w) = 0, \forall w$$

- If the knapsack capacity is 0 ($w=0$), we cannot select any items, so the value is also 0:

$$dp(i, 0) = 0, \forall i$$

Constructing the DP Table

We create a $dp[i][w]$ table where:

- i represents the items.
- w represents the knapsack capacity.

Item	Weight (w_i)	Value (v_i)
1	10	60
2	20	100
3	30	120

Example: Knapsack capacity: $W=50$

DP Table

i\w	0	10	20	30	40	50
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	60	100	160	160	160
3	0	60	100	160	180	220

- **Row 1 (i=1):** Only item 1 ($w_1=10$, $v_1=60$) is considered.
 - If $w < 10$, item cannot be chosen $\rightarrow dp(1,w)=0$.
 - If $w \geq 10$, item 1 is selected $\rightarrow dp(1,w)=60$.
- **Row 2 (i=2):** Adding item 2 ($w_2=20$, $v_2=100$) (consider both item1 and item2).
 - If $w < 20$, item cannot be chosen $\rightarrow dp(2,w)=dp(1,w)$.
 - If $w \geq 20$, we choose the maximum between selecting or not selecting item 2.
- **Row 3 (i=3):** Adding item 3 ($w_3=30$, $v_3=120$) (consider all item 1, item2 and item3).
 - If $w < 30$, item cannot be chosen $\rightarrow dp(3,w)=dp(2,w)$.
 - If $w \geq 30$, we choose the maximum between selecting or not selecting item 3.

Final result: The maximum possible value is 220.

Backtracking to find selected items

We trace back the selected items by checking if:

$$dp(i,w) \neq dp(i-1,w)$$

Backtracking steps:

- Start at $dp(3,50)=220$.
 - Compare with $dp(2,50)=160$.

- Since $220 \neq 160$, item 3 is selected.
- Update remaining weight: $w=50-30=20$.
- Check $dp(2,20)=100$ vs $dp(1,20)=60$.
 - Since $100 \neq 60$, item 2 is selected.
 - Update remaining weight: $w=20-20=0$.
- Stop (weight is 0). Item 1 is not selected.

Selected items:

- Item 3 (Weight = 30, Value = 120)
- Item 2 (Weight = 20, Value = 100)

Total value = 220

Top - down approach

```

KS-MEMO(n, W, memo, w, v)
1. If W == 0 or n == 0
    return 0
2. If memo[n][W] is not -1
    return memo[n][W]
3. If w[n] > W
    result = KS-MEMO(n-1, W, memo) // Cannot include item n
4. Else
    result = max(
        KS-MEMO(n-1, W, memo, w, v),           // Exclude item n
        v[n] + KS-MEMO(n-1, W - w[n], memo, w, v) // Include item n
    )
5. memo[n][W] = result
6. return result
  
```

Bottom - up approach

```

Initialize w[1:n], v[1:n]
KS-TABULATION(n, W, w, v)
  
```

```

1 let dp[0:n][0:W] be a new array
2 for i = 0 to n
3   for w = 0 to W
4     if i == 0 or w == 0
5       dp[i][w] = 0
6     else if w[i] <= W
7       dp[i][w] = max(v[i] + dp[i-1][W-w[i]], dp[i-1][W])
8     else dp[i][w] = dp[i-1][W]
9 return dp[n][W]

```

Reconstructing a solution

```

TRACEBACK(n, W, memo, w, v)
1. items = [] // Stores selected items
2. while n > 0 and W > 0:
3.   If memo[n][W] == memo[n-1][W]:
4.     // Item n was NOT included, move to previous item
5.     n = n - 1
6.   Else:
7.     // Item n was included, add to list
8.     items.append(n)
9.     W = W - w[n] // Reduce remaining capacity
10.    n = n - 1
11. return items // List of selected items

```

3.3. Longest Common Subsequence

Problem:

- Given two strings X and Y, we need to find the longest common subsequence (LCS) between them.
- Subsequence: A sequence derived from another sequence by deleting some characters without changing the order of the remaining characters.
- Longest Common Subsequence (LCS): The longest sequence that appears in both given strings.

Example

X = "ACDBE"

Y = "ABCDE"

The LCS of these two strings is "ACDE", with a length of 4.

State Definition

Define the state function $dp(i, j)$:

$dp(i, j)$ represents the length of the longest common subsequence between:

- The first i characters of string $X \rightarrow X[1..i]$
- The first j characters of string $Y \rightarrow Y[1..j]$

Recurrence Relation

Case 1: When the characters match

If $X[i] == Y[j]$, then we include this character in the LCS:

$$dp(i, j) = dp(i-1, j-1) + 1$$

This means we add $X[i]$ (or $Y[j]$) to the LCS and consider the smaller subproblem.

Case 2: When the characters do not match

If $X[i] != Y[j]$, we take the best LCS possible by either:

- Ignoring the current character of X ($dp(i-1, j)$)
- Ignoring the current character of Y ($dp(i, j-1)$)

$$dp(i, j) = \max(dp(i-1, j), dp(i, j-1))$$

We choose the option that gives the longer LCS.

Example

For $X = "ACDBE"$ and $Y = "ABCDE"$, the dp table is:

	0	A	B	C	D	E
0	0	0	0	0	0	0
A	0	1	1	1	1	1
C	0	1	1	2	2	2
D	0	1	1	2	3	3

B	0	1	2	2	3	3
E	0	1	2	2	3	4

Final LCS length: $dp(5,5) = 4$

Final LCS: "ACDE"

Traceback Rules

After constructing the dp table, we need to trace back to find the actual LCS.

General rules for traceback:

- If $X[i] == Y[j]$, this character is part of the LCS $\rightarrow$ move diagonally (up-left) to $dp(i-1, j-1)$.
- If $X[i-1] != Y[j-1]$, move in the direction with the larger dp value:
- Move up ($dp(i-1, j)$) if $dp(i-1, j) > dp(i, j-1)$.
- Move left ($dp(i, j-1)$) otherwise

Step 1: Start at $dp(5,5)$

- $X[5] = 'E', Y[5] = 'E' \rightarrow$ Characters match.
- Include 'E' in the LCS and move diagonally up-left: $dp(4,4) \rightarrow "E"$

Step 2: Look at $dp(4,4)$

- $X[4] = 'B', Y[4] = 'D' \rightarrow$ Characters do not match.
- Compare $dp(3,4) = 3$ and $dp(4,3) = 2$, choose the higher value $\rightarrow$ Move up: $dp(3,4) \rightarrow "E"$

Step 3: Look at $dp(3,4)$

- $X[3] = 'D', Y[4] = 'D' \rightarrow$ Characters match.
- Include 'D' and move diagonally up-left: $dp(2,3) \rightarrow "DE"$

Step 4: Look at $dp(2,3)$

- $X[2] = 'C', Y[3] = 'C' \rightarrow$ Characters match.
- Include 'C' and move diagonally up-left: $dp(1,1) \rightarrow "CDE"$

Step 5: Look at $dp(1,1)$

- $X[1] = 'A', Y[1] = 'A' \rightarrow$ Characters match.
- Include 'A' and move diagonally up-left: $dp(0,0) \rightarrow "ACDE"$

Top - down approach

LCS-Length(X, Y)

1. $n = \text{length}(X)$ // Get length of X
2. $m = \text{length}(Y)$ // Get length of Y
3. Create table $M[1:n, 1:m]$ initialized with -1 // Memoization table
4. return LCS-recur(X, Y, M, n, m) // Compute LCS length

LCS-recur(X, Y, M, i, j)

5. if $(i == 0 \text{ or } j == 0)$ return 0 // Base case: empty sequence
6. if $(M[i, j] \neq -1)$ return $M[i, j]$ // Return if already computed
7. if $(X[i] == Y[j])$ // Match found (1-based index)
8. $M[i, j] = \text{LCS-recur}(X, Y, M, i-1, j-1) + 1$
9. else
10. $M[i, j] = \max(\text{LCS-recur}(X, Y, M, i-1, j), \text{LCS-recur}(X, Y, M, i, j-1))$
11. return $M[i, j]$

Bottom - up approach

LCS-BottomUp(X, Y)

1. $n = \text{length}(X)$ // Get length of X
2. $m = \text{length}(Y)$ // Get length of Y
3. Create table $M[0:n, 0:m]$ initialized with 0 // DP table
4. for $i = 1$ to n
5. for $j = 1$ to m
6. if $X[i] == Y[j]$ // Match found
7. $M[i, j] = M[i-1, j-1] + 1$
8. else
9. $M[i, j] = \max(M[i-1, j], M[i, j-1])$
10. return $M[n, m]$ // LCS length

Reconstructing a solution

LCS-Traceback(X, Y, M)

1. $i = \text{length}(X)$
2. $j = \text{length}(Y)$
3. $\text{LCS_sequence} = ""$ // Empty string to store LCS
4. while $i > 0$ and $j > 0$
5. if $X[i] == Y[j]$ // Match found, move diagonally
6. Prepend $X[i]$ to LCS_sequence
7. $i = i - 1$
8. $j = j - 1$
9. else if $M[i-1, j] > M[i, j-1]$ // Move up

```
10.    i = i - 1
11.  else // Move left
12.    j = j - 1
13. return LCS_sequence // The LCS found
```